Сценарий демонстрации

Английский оригинал: demo.md.

Проход по инструменту на эталонном корпусе на десять–пятнадцать минут. Каждое число ниже производится кодом на захватах в `tests/corpus/`; ничего не рассказывается, чего инструмент не показывает. Окно было проведено через этот сценарий до того, как он записан, и то, что оно показывает, зафиксировано в `docs/GUI_VERIFICATION.md`; то же исследование без окна — `docs/demo_cli.md`.

Подготовка

Окружение:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

Файлы, которые открывает проход:

- `tests/corpus/reference_export/corpus_capture_01.normalized.json` — первый захват, нормализованный.
- `tests/corpus/reference_export/corpus_capture_02.normalized.json` — второй захват того же протокола.
- `examples/corpus_rule_v1.json` — первое правило, слишком узкое по коду команды.
- `examples/corpus_rule_v2.json` — уточнённое правило.

Запуск окна:

```
python -m src.ui.main_window \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --rule examples/corpus_rule_v1.json
```

Окно открывает тот захват и правило, которые вы передали в командной строке или через «Файл, Открыть захват» и «Файл, Открыть правило»; встроенного примера нет.

Перед проходом убедитесь, что весь набор зелёный:

```
QT_QPA_PLATFORM=offscreen pytest tests/ -q
```

Шаг 1 — открыть захват (1 мин)

Делаем: «Файл, Открыть захват», выбираем `corpus_capture_01.normalized.json`.

Видим: три сессии в дереве, каждая со своими двумя направлениями и счётчиками байт. Строка состояния показывает id захвата и диагностики.

Говорим: инструмент читает нормализованный захват, а не pcap. Этим шагом владеет модуль захвата; здесь байты уже собраны, и каждый диапазон помнит свой пакет.

Шаг 2 — провенанс байта (1 мин)

Делаем: выбираем сессию `s2`, направление `A_to_B`. Ведём указатель по первым байтам.

Видим: строка состояния показывает сессию, направление, смещение и индекс пакета, номер последовательности и время байта.

Говорим: байт привязан к пакету, из которого пришёл. Утверждение о байте всегда можно проверить по захвату.

Шаг 3 — применить первое правило (2 мин)

Делаем: «Файл, Открыть правило», `corpus_rule_v1.json`. Затем «Правило, Применить к текущему направлению» или F5.

Видим: вкладка «Интерпретация» перечисляет сообщения с их статусом. Вкладка контрпримеров наполняется; строка состояния читает версию правила и счётчики.

Говорим: правило фреймит каждое сообщение из одного описания; сообщения не перечисляются вручную. Зелёный — matched, красный — контрпример.

Шаг 4 — контрпример и его байты (2 мин)

Делаем: открываем вкладку «Контрпримеры», выбираем первую запись.

Видим: hex-просмотр прокручивается к сообщению, а offending байты подсвечены; строка состояния показывает их пакет и номер последовательности.

Говорим: этот запрос несёт команду `2`, а правило v1 допускало только `1`. Контрпример привязан к реальным байтам, а не к их описанию.

Шаг 4b — альтернативные чтения (1 мин)

Делаем: «Правило, Применить ко всему захвату» или F6, оставив правило v1. Затем открываем вкладку «Гипотезы».

Видим: по одной строке на каждого кандидата-чтения каждого поля `hypothesis`, с поддержкой, числом противоречий и баллом. Для `command` `entropy_enum` и `low_cardinality` возглавляют список с 1.00 (140 поддержки, 0 противоречий), а `constant` — чтение, которое объявляет правило v1, — стоит на 0.43 (60 поддержки, 80 противоречий).

Говорим: прежде чем расширять правило, поле отдаётся движку альтернатив по каждому сообщению, к которому правило применимо. Чтение, которое правило уже объявляет, — лишь один кандидат, и на этом захвате большинство сообщений ему противоречат. Это доказательство за уточнением, а не догадка. Выбор строки прыгает hex-просмотром к сообщению, которое её поддерживает или опровергает.

Шаг 5 — уточнить правило (2 мин)

Делаем: «Файл, Открыть правило», `corpus_rule_v2.json`. Снова применяем к `s2`, `A_to_B`.

Видим: список контрпримеров пуст; каждое сообщение matched.

Говорим: уточнение расширило допустимые коды команды до `1`, `2` и `3`. Структура правила не изменилась. Прежний результат остаётся на диске со своей старой версией и помечается outdated.

Шаг 6 — проверка по всему захвату (1 мин)

Делаем: «Правило, Применить ко всему захвату» или F6.

Видим: строка состояния сообщает счётчики по каждому направлению: `matched=140, mismatched=0, ...`.

Говорим: правило проверяется по всему корпусу, а не по одному сообщению. Правило v1 дало 60 matched и 80 контрпримеров; правило v2 даёт 140 matched и ни одного.

Шаг 7 — перенос на второй захват (1 мин)

Делаем: «Файл, Открыть захват», `corpus_capture_02.normalized.json`. Применяем ко всему захвату правилом v2.

Видим: 40 matched, 0 контрпримеров, 40 not applicable (ответы, которых правило не охватывает).

Говорим: правило не подгонялось под этот захват, и всё равно держится. Это доказательство общности внутри этого протокола.

Шаг 8 — граница применимости (2 мин)

Делаем: синтетический захват не поставляется как нормализованная выгрузка, потому что он вне эталонного корпуса. Сначала нормализуем его, затем открываем:

```
python -m src.capture.cli --pcap tests/corpus/synthetic_live.pcapng \
  --out /tmp/synthetic_live.normalized.json
```

Затем «Файл, Открыть захват», `/tmp/synthetic_live.normalized.json`. Применяем ко всему захвату правилом v2.

Видим: ни одно сообщение не совпадает; фрейминг сообщает поток как incomplete или not applicable, поэтому набор контрпримеров не пуст, а счётчик совпадений равен нулю.

Говорим: другой захват использует little-endian длину, id транзакции и другие коды команд. Правило не переносится. Инструмент сообщает о сбое вместо того, чтобы его скрыть. Это граница области применимости правила.

Шаг 9 — корреляция с журналом (1 мин)

Делаем: эта корреляция показывается скриптом исследования, а не окном. Запускаем:

```
python -m scripts.run_reference_investigation
```

Видим: раздел журнала сообщает 140 из 140 запросов, скоррелированных на первом захвате, и 40 из 40 на втором, с раскодированной целью, равной параметру журнала на каждом запросе.

Говорим: независимая запись извне байтов согласуется с чтением поля параметра. Это доказательство, а не доказанность.

Шаг 10 — отчёт и переносной проект (1 мин)

Делаем: «Отчёт, Показать REPORT.md». Затем «Отчёт, Экспорт Markdown», сохраняем в файл.

Видим: отчёт на панели и выгруженный файл.

Говорим: исследование изложено с его контрпримерами, версиями и пределами. Каталог проекта можно скопировать в другое место и снова открыть.

Переносной проект ведётся из меню проекта в окне, а также из командной строки, когда предпочтителен скрипт. Чтобы показать путь CLI, запустите:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project /tmp/demo.madrigal --rule examples/corpus_rule_v2.json
python -m src.project.cli export --project /tmp/demo.madrigal --out /tmp/demo.zip
python -m src.project.cli import --in /tmp/demo.zip --target /tmp/demo.moved
```

Видим: конвейер пишет проект с захватом, нормализованным JSON и отрисованным отчётом; круговой прогон экспорта и импорта сохраняет контрольную сумму захвата. Полный проход по командной строке — `docs/demo_cli.md`.

Резерв — командная строка

Если окно не может работать на демонстрационной машине, тот же результат даёт командная строка:

```
QT_QPA_PLATFORM=offscreen python -m src.protocol.cli apply \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/result.json

QT_QPA_PLATFORM=offscreen python -m src.protocol.cli verify \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json

QT_QPA_PLATFORM=offscreen python -m scripts.run_reference_investigation
```

Команда `apply` пишет результат, проходящий проверку по `docs/schemas/result.schema.json`. Команда `verify` печатает счётчики и контрпримеры. Скрипт исследования пересобирает полное изложение.

Тайминг

шаг	минут
1 открыть захват	1
2 провенанс	1
3 применить v1	2
4 контрпример	2
4b альтернативы	1
5 уточнить	2
6 весь захват	1
7 перенос	1
8 граница	2
9 журнал	1
10 отчёт	1
итого	15